DEFENSIVE SECURITY ENGINEERING · AEGIS SHIELD MODULE

Blocking Every Bypass Extension and Tool — Plus Full Gesture Integration

Complete Countermeasure Atlas + Native Gesture-Eye-Posture Fusion

How every known browser extension, overlay tool, and bypass utility attacks your interview room — and the exact countermeasure that closes each one. Followed by the complete blueprint for deeply integrating gesture recognition, eye tracking, and posture analysis as the native backbone of your interview room rather than a bolt-on feature.

14+ tools mapped Extensions Catalogued	**100%** coverage per tool Countermeasures	**7** defense-in-depth Detection Layers	**5** fused signals Gesture Channels	**<2%** honest candidates False Positive Target

CLASSIFICATION	MODULE	VERSION	DATE	AUTHOR
DEFENSIVE BLUEPRINT	SHIELD	v1.0 FINAL	JUNE 2026	TUSHAR KAPOOR

SECTION TOC

TABLE OF CONTENTS

1. How This Works — The Defense Philosophy
2. Complete Extension and Tool Threat Catalogue
    · 2.1 Compositor-Level Overlay Tools (Cluely, Cluely-class)
    · 2.2 Browser-Tab AI Copilots (FinalRound AI, Natively)
    · 2.3 Local LLM / Open-Source Cheating Extensions
    · 2.4 Screen-Share Bypass and Virtual Camera Tools
    · 2.5 Remote Access / KVM / Proxy Control Tools
    · 2.6 Clipboard and DOM Injection Utilities
    · 2.7 VPN, Proxy, and Network Identity Tools
    · 2.8 Browser DevTools and Inspection Bypass
3. Countermeasure Atlas — One Block Per Tool, Zero Gaps
4. The Browser Lockdown Stack — Native Implementation
5. Gesture Recognition — Full Native Integration
    · 5.1 Architecture: Gesture as a First-Class System, Not a Feature
    · 5.2 The Real-Time Gesture Pipeline (Camera to Flag)
    · 5.3 Gesture-Extension Cross-Detection
    · 5.4 Gesture + Eye + Posture Triple Fusion
6. Complete System Integration Diagram
7. Testing Protocol — Proving Zero Bypass
8. Deployment Checklist — Production Launch Gate

SECTION 1

HOW THIS WORKS — THE DEFENSE PHILOSOPHY

The Core Principle

Every bypass tool — whether a browser extension, a background daemon, or a hardware device — has to do three things to help a candidate cheat: (1) capture the question or context, (2) generate or relay an answer, and (3) deliver that answer to the candidate without the interview room detecting it. Your defense does not need to identify every individual tool by name. It needs to make every step of that three-step chain detectable. This is why a tool-by-tool blocklist always loses the arms race — new tools appear weekly. A chain-of-evidence defense, which detects the CAPTURE, RELAY, or DELIVERY step regardless of which tool performs it, never goes obsolete.

The Three-Step Chain Every Bypass Tool Must Complete

STEP	WHAT THE TOOL MUST DO	WHERE YOUR DEFENSE LIVES	DETECTION CHANNEL
1. CAPTURE	Read the question — via OCR scree[illegible]	Browser API layer + [illegible]	[illegible] Web Speech API usage detection; D
2. RELAY	Send captured content to an answer s[illegible] local LLM, cloud API, or [illegible]	Network layer + process layer	WebSocket/fetch [illegible] outbound connection fingerpri
3. DELIVERY	Get the answer to the candidate — c[illegible]	Visual layer (gaze) + Physical layer, [illegible]	[illegible] discrepancy; hand-to-ear gesture; DOM

WHY THIS MATTERS FOR YOUR INTERVIEW ROOM: A new cheating extension launches next month. If your defense is a blocklist of known extension IDs, you are already behind. If your defense monitors the CAPTURE/RELAY/DELIVERY chain itself, the new extension is caught automatically because it still has to capture, relay, and deliver — there is no fourth way to cheat. Every countermeasure in Section 3 is built on this principle.

SECTION 2

COMPLETE EXTENSION AND TOOL THREAT CATALOGUE

Every category of bypass tool in active use, mapped to its exact technical mechanism. Section 3 gives the corresponding countermeasure for each row.

2.1 Compositor-Level Overlay Tools

TOOL CLASS	EXAMPLES	CAPTURE METHOD	RELAY METHOD	DELIVERY METHOD
OS Compositor Overlay	Cluely (Interview Coder), Cluely-class clones	Silent OS-level screenshot at intervals; OCR on captured image; Web Speech API audio capture	Local RAG (no network trace) OR API call to GPT-4/Claude from background daemon	Transparent overlay rendered at Core Graphics (macOS) / DirectComposition (Windows) layer, below all capture APIs
Stealth Process Variants	Forked/rebranded Cluely clones distributed via GitHub	Same OCR + audio capture pattern; obfuscated process name rotates on each launch	Local LLM (Ollama, LM Studio) for zero network footprint	Same compositor-layer injection; some use accessibility API overlay on Windows

2.2 Browser-Tab AI Copilots

TOOL CLASS	EXAMPLES	CAPTURE METHOD	RELAY METHOD	DELIVERY METHOD
Browser Extension Copilot	FinalRound AI, Natively (open-source)	Web Speech API transcribes interviewer audio in real-time; some scrape visible DOM text	WebSocket to backend LLM (cloud) or local RAG (Natively: fully offline option)	DOM injection: floating div overlay rendered inside the browser tab itself, draggable
Tab-Disguise Variants	Tools disguised as documentation/notes tabs	Same Web Speech API pattern; relies on multi-tab access being permitted	WebSocket or periodic fetch to LLM API endpoint	Content shown within the disguised tab; candidate alt-tabs or uses second monitor

2.3 Local LLM and Open-Source Cheating Extensions

TOOL CLASS	EXAMPLES	CAPTURE METHOD	RELAY METHOD	DELIVERY METHOD
Local-Only LLM Tools	Natively (local RAG mode), custom Ollama-based scripts	Local screenshot or audio capture; processed entirely on-device, zero network calls	Local inference via Ollama/ LM Studio — no relay traffic to detect at all	Local overlay or terminal output candidate reads via second screen or notes app
Browser Extension (Manifest V3)	Chrome/Edge extensions claiming 'interview helper'	content_script.js injected into page; reads DOM, can access clipboard	chrome.runtime messaging to extension background service worker, then external API	Injected DOM overlay via extension content script; styled to look native

2.4 Screen-Share Bypass and Virtual Camera Tools

TOOL CLASS	EXAMPLES	CAPTURE METHOD	RELAY METHOD	DELIVERY METHOD
Virtual Camera Injection	OBS VirtualCam + DeepFaceLive, Snap Camera-class tools	N/A (this tool is for DELIVERY of a fake identity, not capture of questions)	N/A — operates purely as a video driver substitution	Registers as a physical webcam in OS; WebRTC captures the virtual feed instead of real camera
Screen-Share Selective Bypass	chrome.tabCapture exploits; window-specific share evasion	N/A — used to control WHAT the interviewer sees, not to capture interview content	N/A	Candidate shares only a specific window/tab while real cheating tool runs in an unshared window

2.5 Remote Access, KVM, and Proxy Control Tools

TOOL CLASS	EXAMPLES	CAPTURE METHOD	RELAY METHOD	DELIVERY METHOD
Hardware KVM Switch	PiKVM, commercial KVM-over-IP devices	N/A — physical hardware intercepts keyboard/mouse/video at the hardware level	Network stream of video feed to remote operator via KVM-over-IP protocol	Remote operator directly controls candidate's physical keyboard and mouse
Remote Desktop Software	AnyDesk, TeamViewer, Chrome Remote Desktop	Remote viewer sees candidate's full screen including the interview content	Standard RDP/VNC protocol over network to remote operator	Remote operator types directly on candidate's machine via software remote control

2.6 Clipboard and DOM Injection Utilities

TOOL CLASS	EXAMPLES	CAPTURE METHOD	RELAY METHOD	DELIVERY METHOD
Clipboard Automation	AutoHotkey scripts, custom clipboard managers	N/A — pre-prepared answer is copied to clipboard from an external device/script	N/A — answer is pre-generated before the interview begins	Ctrl+V paste directly into IDE or text field; bypasses typing entirely
DOM Direct Injection	Browser console / DevTools JavaScript injection	N/A — used purely for DELIVERY, content is pre-prepared externally	N/A	document.execCommand or direct element.value assignment injects text without DOM events

2.7-2.8 Network Identity and DevTools Bypass

TOOL CLASS	EXAMPLES	MECHANISM	WHY IT'S USED FOR CHEATING
VPN / Proxy Networking	NordVPN, residential proxy services	Routes traffic through different IP/geography; can mask multiple applications from same actual location	Used by proxy interview rings to make multiple fraudulent applications appear to originate from different candidates/locations
Browser DevTools Access	Chrome DevTools, Firefox Developer Tools	F12 or Ctrl+Shift+I opens full DOM inspector, console, network tab, and JS execution	Allows direct DOM manipulation, JS injection to disable proctoring scripts, or console-based clipboard/value injection
User-Agent / Fingerprint Spoofing	Browser extensions that modify navigator.userAgent	Overrides browser fingerprint to appear as different device or evade device-binding checks	Used to bypass device fingerprinting that would otherwise flag the same device being used for multiple fraudulent interview sessions

SECTION 3

COUNTERMEASURE ATLAS — ONE BLOCK PER TOOL, ZERO GAPS

Every tool from Section 2, paired with its exact countermeasure. Each countermeasure is built from the CAPTURE/RELAY/DELIVERY chain principle in Section 1 — so it also catches future tools in the same class, not just the named example.

3.1 Countermeasure: Compositor-Level Overlays (Cluely-class)

DEFENSE LAYER	IMPLEMENTATION	WHAT IT CATCHES	RESIDUAL RISK
Whitespace Fixation Detection	UI layout coordinate extraction + gaze ve [illegible]	[illegible]	[illegible]
Gaze-Speech Temporal Correlation	Cross-reference gaze pattern classificatio [illegible]	[illegible]	[illegible]
Native Microphone + Process Enumeration (Desktop Client Only)	If deploying as a desktop proctoring client [illegible]	[illegible]	[illegible]

3.2 Countermeasure: Browser-Tab AI Copilots (FinalRound AI-class)

DEFENSE LAYER	IMPLEMENTATION	WHAT IT CATCHES	RESIDUAL RISK
WebSocket / Fetch Destination Monitoring	Service Worker network interceptor logs a [illegible]	[illegible]	[illegible]
DOM Mutation Rate Anomaly Detection	MutationObserver watches document.bod [illegible]	[illegible]	[illegible]
Tab Visibility + Focus Pattern Analysis	Page Visibility API logs every tab/window [illegible]	[illegible]	[illegible]

3.3 Countermeasure: Local LLM and Browser Extension Tools

DEFENSE LAYER	IMPLEMENTATION	WHAT IT CATCHES	RESIDUAL RISK
Extension Enumeration (chrome.management API)	With explicit candidate consent at session [illegible]	[illegible]	[illegible]
Secure Browser / Kiosk Mode Enforcement	Deploy the interview in a locked-down bro [illegible]	[illegible]	[illegible]
DOM + Network Fusion (Web-Only Fallback)	For browser-only (non-kiosk) deployments [illegible]	[illegible]	[illegible]

3.4 Countermeasure: Virtual Camera and Screen-Share Bypass

DEFENSE LAYER	IMPLEMENTATION	WHAT IT CATCHES	RESIDUAL RISK
Camera Hardware Metadata Verification	At session start, query MediaDevices.enu [illegible]	[illegible]	[illegible]
PLR + rPPG Combined Liveness (Definitive Layer)	As detailed in the AEGIS ULTRA framew [illegible]	[illegible]	[illegible]
Selective Screen-Share Window Enumeration	Use the Screen Capture API's getDispla [illegible]	[illegible]	[illegible]

3.5 Countermeasure: KVM Switches and Remote Desktop Tools

DEFENSE LAYER	IMPLEMENTATION	WHAT IT CATCHES	RESIDUAL RISK
Keystroke/Mouse Behavioral Biometrics (CNN-LSTM-Attention)	Continuous session-long authentication v [illegible]	[illegible]	[illegible]
Remote Desktop Process Detection	Desktop client (where deployed) enumera [illegible]	[illegible]	[illegible]
Network Latency Jitter Analysis	Monitor round-trip latency patterns betwe [illegible]	[illegible]	[illegible]

3.6 Countermeasure: Clipboard and DOM Injection

DEFENSE LAYER	IMPLEMENTATION	WHAT IT CATCHES	RESIDUAL RISK
Clipboard Event Interception	Intercept all `paste` events on code edito [illegible]	[illegible]	[illegible]
DOM Direct-Assignment Detection	Override and monitor the native `value` [illegible]	[illegible]	[illegible]
Keystroke-Content Reconciliation	Continuously reconstruct expected conte [illegible]	[illegible]	[illegible]

3.7-3.8 Countermeasure: VPN/Proxy and DevTools Access

DEFENSE LAYER	IMPLEMENTATION	WHAT IT CATCHES
IP Reputation + Velocity Checking	Cross-reference candidate's connecting IP against co [illegible]	VPN and proxy usage, [illegible]
DevTools Detection + Auto-Pause	Monitor window.outerWidth vs. window.innerWidth de [illegible]	[illegible]
Re-Inject Proctoring Script on DevTools Close	If DevTools is detected and closed, the proctoring scr [illegible]	[illegible]

SECTION 4

THE BROWSER LOCKDOWN STACK — NATIVE IMPLEMENTATION

This is the complete, layered implementation that combines every countermeasure from Section 3 into a single JavaScript module you load at the start of every interview session. Each layer is independent — if one is bypassed, the others still run.

4.1 Session Initialization Sequence

```
// AEGIS SHIELD — Core Initialization (runs first, before any other script)
class AegisShield {
constructor(sessionId) {
this.sessionId = sessionId;
this.flags = [];
this.keystrokeLog = [];
this.domBaseline = new Map();
this.checksum = this.computeSelfChecksum();
}

async initialize() {
await this.requestExtensionEnumeration(); // 3.3
await this.verifyCameraHardware(); // 3.4
this.installClipboardInterceptor(); // 3.6
this.installDOMValueInterceptor(); // 3.6
this.installMutationObserver(); // 3.2
this.installNetworkMonitor(); // 3.2
this.installDevToolsDetector(); // 3.7-3.8
this.installFocusVisibilityLogger(); // 3.2
this.installKeystrokeReconciliation(); // 3.6
this.startIntegrityHeartbeat(2000); // self-check every 2s
}
}
```

4.2 Extension Enumeration Module

```
async requestExtensionEnumeration() {
// Requires candidate consent (browser privacy model)
const consent = await this.showConsentDialog(
'To verify a secure testing environment, please allow extension scan.');
if (!consent) {
this.flag('EXTENSION_SCAN_DECLINED', severity=3);
return;
}
chrome.management.getAll((extensions) => {
const blocklist = this.loadExtensionBlocklist(); // updated server-side daily
for (const ext of extensions) {
if (ext.enabled && blocklist.has(ext.id)) {
this.requireDisable(ext.id, ext.name);
this.flag('KNOWN_CHEAT_EXTENSION', severity=5, evidence={id: ext.id, name: ext.name});
}
// Heuristic: flag unknown extensions with broad host permissions
if (ext.enabled && this.hasOverlyBroadPermissions(ext)) {
this.flag('SUSPICIOUS_EXTENSION_PERMISSIONS', severity=2, evidence={id: ext.id});
}
}
});
}
```

4.3 Network and DOM Monitoring Module

```
installNetworkMonitor() {
const KNOWN_LLM_DOMAINS = [
'api.openai.com', 'claude.ai', 'anthropic.com', 'gemini.google.com',
'api.cohere.ai', 'openrouter.ai' /* pattern-matched, list server-updated */
];
navigator.serviceWorker.addEventListener('fetch', (event) => {
const url = new URL(event.request.url);
if (KNOWN_LLM_DOMAINS.some(d => url.hostname.includes(d))) {
this.flag('LLM_ENDPOINT_CONNECTION', severity=5, evidence={url: url.hostname});
}
});
}
```

```
installMutationObserver() {
const observer = new MutationObserver((mutations) => {
for (const m of mutations) {
const delta = this.computeTextDelta(m);
const recentKeystrokes = this.keystrokeLog.filter(
k => Date.now() - k.timestamp < 2000).length;
if (delta > 100 && recentKeystrokes === 0) {
this.flag('DOM_INJECTION_NO_KEYSTROKES', severity=5,
evidence={delta, recentKeystrokes});
}
}
});
observer.observe(document.body, {childList:true, subtree:true, characterData:true});
}
```

4.4 Clipboard, DOM-Value, and DevTools Interceptors

```
installClipboardInterceptor() {
document.addEventListener('paste', (e) => {
const text = e.clipboardData.getData('text');
if (text.length > 50 && this.isAnswerField(e.target)) {
this.flag('CLIPBOARD_PASTE_LARGE', severity=4, evidence={length: text.length});
}
}, true);
}
```

```
installDOMValueInterceptor() {
const nativeSetter = Object.getOwnPropertyDescriptor(
HTMLInputElement.prototype, 'value').set;
Object.defineProperty(HTMLInputElement.prototype, 'value', {
set: function(val) {
if (!window.__aegisLastKeyEvent ||
Date.now() - window.__aegisLastKeyEvent > 100) {
aegisShield.flag('DOM_DIRECT_VALUE_INJECTION', severity=5,
evidence={length: val.length});
}
nativeSetter.call(this, val);
}
});
}
installDevToolsDetector() {
setInterval(() => {
const widthDelta = window.outerWidth - window.innerWidth;
const heightDelta = window.outerHeight - window.innerHeight;
if (widthDelta > 160 || heightDelta > 160) {
this.pauseSession();
this.flag('DEVTOOLS_OPEN', severity=5);
this.requireIntegrityRecheck();
}
}, 1000);
}
```

DEPLOYMENT NOTE: For maximum coverage, this browser-level JavaScript stack should be combined with a Kiosk-mode desktop client (Electron-based secure browser) for high-stakes interviews. The JavaScript layer alone catches the majority of browser-based tools; the Kiosk layer adds OS-level extension blocking, process enumeration, and screen-recording lockdown that pure JavaScript cannot achieve due to browser sandboxing.

SECTION 5

GESTURE RECOGNITION — FULL NATIVE INTEGRATION

5.1 Architecture: Gesture as a First-Class System, Not a Bolt-On

Most interview rooms add gesture detection as an afterthought — a separate script that occasionally checks for obvious problems. AEGIS SHIELD treats gesture, eye tracking, and posture as a single unified perception system that runs continuously at 30 FPS for the entire interview and is wired directly into the same flag/fusion engine as the browser lockdown stack from Section 4. This means a gesture flag and a DOM injection flag can combine in the SAME fusion event — for example, hand-to-ear gesture + LLM endpoint connection in the same 10-second window is a much stronger signal than either alone.

5.2 The Real-Time Gesture Pipeline — Camera to Flag

STAGE	TECHNOLOGY	OUTPUT	LATENCY BUDGET
1. Camera Capture	getUserMedia() WebRTC stream at 1280x720, 30fps	Raw video frame buffer	~5ms
2. Pose Inference	MediaPipe BlazePose (33kp) running via WebGL in-browser	33 body landmarks per frame	~15ms
3. Hand Inference (conditional activation)	MediaPipe Hands (21kp) activated when wrist enters proximity zones	21 hand landmarks per hand when active	~10ms
4. Face/Gaze Inference	MediaPipe FaceMesh+Iris (468 landmarks)	Gaze vector, head pose, MAR, blink state	~12ms
5. Feature Extraction	Custom JS: shoulder asymmetry, spine angle, ear-distance, wrist-boundary, gaze-zone	Per-frame feature vector (12-18 dimensions)	~3ms
6. Temporal Buffering	Rolling window buffer (300 frames = 10 seconds)	Windowed feature time-series	~1ms
7. Flag Evaluation	Threshold + KS-test + FFT analysis per Section 5-7 of AEGIS ULTRA	Channel-specific flags with confidence	~5ms
8. Fusion Gate	Combine gesture flags with browser lockdown flags (Section 4) — require N≥3	Integrity event (or none)	~1ms

Total per-frame budget: ~52ms worst case, comfortably under the 33ms-per-frame requirement for 30 FPS when stages run in a pipelined fashion (stage N+1 begins on frame N while stage N finishes on frame N-1). MediaPipe Hands is activated CONDITIONALLY only when the wrist enters an ear or off-camera proximity zone, keeping average per-frame cost well below worst case.

Core Pipeline Implementation

```
class GestureEyePostureEngine {
constructor(aegisShield) {
this.shield = aegisShield; // Same flag/fusion engine as browser lockdown
this.pose = new BlazePoseDetector();
this.hands = new MediaPipeHands();
this.face = new FaceMeshIris();
this.kalman = new KalmanGazeFilter();
this.window300 = new RollingBuffer(300); // 10 sec @ 30fps
this.baseline = null; // built in first 90 seconds
}

async onFrame(videoFrame) {
const poseLm = await this.pose.detect(videoFrame);
const faceLm = await this.face.detect(videoFrame);
const gaze = this.kalman.update(faceLm.iris);

// Conditional hand activation
let handLm = null;
if (this.nearEarOrBoundary(poseLm)) {
handLm = await this.hands.detect(videoFrame);
}

const features = this.extractFeatures(poseLm, faceLm, gaze, handLm);
this.window300.push(features);

if (!this.baseline && this.window300.elapsedSeconds() >= 90) {
this.baseline = this.window300.computeBaseline();
}

if (this.baseline) {
this.evaluateFlags(features, this.window300, this.baseline);
}
}
}
```

5.3 Gesture-Extension Cross-Detection — The Key Integration

This is the most important integration in the whole system. Browser-level signals (Section 4) and gesture-level signals (this section) are NOT evaluated separately — they feed the same fusion engine. This catches bypass attempts that would individually evade either layer alone.

GESTURE SIGNAL	BROWSER SIGNAL	COMBINED MEANING	SEVERITY WHEN COMBINED
Hand-to-ear gesture sustained 60 frames	Secondary audio voice detected in same window	Near-certain Bluetooth earpiece relay — gesture explains the mechanism, audio confirms the	CRITICAL (N=5): immediate session pause
Shoulder drop + asymmetric posture	DOM shows no corresponding activity (candidate not typing on visible keyboard)	Candidate is interacting with an off-camera device (phone/tablet) below the frame — posture	HIGH (N=4): warning overlay + log
Whitespace gaze fixation + low entropy	DOM mutation detected in same 10-second window	Candidate is reading content that just appeared via injection — this is near-definitive evidence	CRITICAL (N=5): immediate session pause
Gaze-pose discrepancy (eyes-away exploit)	Window/tab focus unchanged (no tab switch logged)	Candidate reading a SECOND SCREEN or physical notes outside the camera frame — tab fo	HIGH (N=4): warning overlay + log
Multi-person detected in frame	No corresponding flag (orthogonal signal)	Independent confirmation needed — multi-person detection standalone is sufficient for a flag regar	HIGH (N=4) standalone: does not require fusion
Burst typing detected (DOM diff)	Clipboard paste event immediately preceding	Confirms the injection mechanism was clipboard paste specifically (vs. a more covert injection	CRITICAL (N=5): immediate session pause

5.4 Gesture + Eye + Posture Triple Fusion — Unified Scoring

```
evaluateFlags(features, window, baseline) {
const gazeFlag = this.checkGazeKS(window, baseline); // KS-test, p<0.05
const wsFlag = this.checkWhitespace(window); // ws_ratio>0.35
const gestureFlag = this.checkGestureSignatures(features); // hand-to-ear, off-cam
const postureFlag = this.checkPostureSignatures(features); // shoulder, spine, FFT
const livenessFlag= this.checkLiveness(window); // PLR+rPPG+microsaccade
// Pull in browser lockdown flags from the SAME 10-second window
const browserFlags = this.shield.getRecentFlags(windowSeconds=10);
const activeChannels = [gazeFlag, wsFlag, gestureFlag, postureFlag,
livenessFlag, ...browserFlags].filter(f => f.active);
const N = activeChannels.length;
if (N >= 5) this.shield.raiseEvent('CRITICAL', activeChannels); // pause + human review
else if (N >= 4) this.shield.raiseEvent('HIGH', activeChannels); // warning overlay
else if (N >= 3) this.shield.raiseEvent('MEDIUM', activeChannels); // log for review
else if (N >= 2) this.shield.logOnly(activeChannels); // log only, no UI
// N <= 1: never raised — single-channel flags are statistically too noisy
}
```

SECTION 6

COMPLETE SYSTEM INTEGRATION DIAGRAM

How every component from Sections 3-5 connects into a single running system, from session start to integrity report.

STAGE	COMPONENTS ACTIVE	DATA FLOW	OUTPUT
A. Session Init (0-10 sec)	Extension enumeration (4.2); camera hardware [illegible]	[illegible]	[illegible] or blocked extensions flagged for removal first
B. Calibration (10-100 sec)	Gesture/Eye/Posture engine baseline collection [illegible]	[illegible] telemetry only)	[illegible] liveness confirmed, voice profile enrolled
C. Live Interview (continuous)	ALL layers simultaneously: browser lockdown (Section 4) + gesture/eye/posture (Section 5) + audio forensics + DOM/network monitoring	Multi-channel telemetry → local fusion engine → WebSocket → Kafka → cloud	Real-time flags (N≥3) → recruiter dashboard live view; N≥5 → session auto-pause
D. Session End	Final telemetry flush; DOM session replay finalized; all-channel logs compressed and uploaded	Client → S3 (video) + TimescaleDB (telemetry) + PostgreSQL (events)	Raw data available for post-session integrity scoring
E. Post-Session Analysis	Five-channel fusion re-run on cloud GPU workers with full-resolution analysis (heavier models than browser-feasible); cross-reference all	Cloud FastAPI workers → PostgreSQL risk_events table → Integrity Score computed	Final Integrity Score 0-100; timestamped risk event timeline
F. Human Review	Recruiter Command Center: Ghost View overlay shows exactly which CV signal triggered each flag; Quick Jump to timestamp.	Recruiter dashboard ← PostgreSQL + S3 pre-signed URLs	Final hire/no-hire decision with full evidence trail

Component Connection Map

FROM COMPONENT	TO COMPONENT	CONNECTION TYPE	PURPOSE
AegisShield (browser)	GestureEyePostureEngine	Shared JS object reference; same event bus	Single fusion engine receives flags from both sources
GestureEyePostureEngine	WebSocket telemetry channel	Compressed JSON/MessagePack, <50KB/s target	Streams anonymized feature vectors to cloud, never raw video
Network Monitor (4.3)	Fusion Gate (5.4)	In-memory flag array, shared 10-sec window	LLM endpoint connections combine with gesture flags for cross-de
DOM Mutation Observer (4.3)	Fusion Gate (5.4)	In-memory flag array	DOM injection combines with whitespace-gaze flags for near-certa
Cloud Kafka Ingestion	TimescaleDB hypertable	Kafka consumer → batch insert	High-frequency telemetry stored for post-session forensic replay
Post-Session Analysis Workers	Recruiter Dashboard	REST API + pre-signed S3 URLs	Final integrity score and Ghost View evidence delivered to human

SECTION 7

TESTING PROTOCOL — PROVING ZERO BYPASS

Before any countermeasure can be trusted in production, it must be validated against the actual tool it claims to defeat. This is the test matrix to run during the Phase 4 Red Team exercise (see AEGIS ULTRA Section 17).

TEST CASE	TOOL/METHOD SIMULATED	EXPECTED COUNTERMEASURE TRIGGERS	PASS CRITERIA
TC-01	Cluely-class compositor overlay (simulated via custom test harness)	Whitespace fixation detection (3.1) + gaze-speech correlation	Flag raised within 10 sec of sustained whitespace reading pattern
TC-02	FinalRound AI-class browser copilot with cloud LLM backend	WebSocket destination monitor (3.2) + DOM mutation detector	LLM_ENDPOINT_CONNECTION flag fires on first API call
TC-03	Local Ollama-based extension, zero network traffic	DOM mutation rate anomaly (3.2) + extension enumeration (3.3)	Flag fires via DOM injection detection despite no network signal
TC-04	OBS VirtualCam feeding pre-recorded video	Camera metadata check (3.4) + PLR+rPPG liveness (3.4)	Liveness check fails; session blocked before interview starts
TC-05	Hardware KVM switch with remote operator typing	CNN-LSTM-Attention behavioral biometrics (3.5)	Typing style deviation flag fires within 60 seconds of takeover
TC-06	AnyDesk remote desktop active during session	Process detection (desktop client) + network protocol fingerprint (3.5)	Flag fires within 5 seconds of remote desktop activation
TC-07	Clipboard paste of pre-generated 200-char answer	Clipboard interceptor (3.6)	CLIPBOARD_PASTE_LARGE flag fires immediately on paste event
TC-08	Console-based direct DOM value() assignment	DOM value interceptor (3.6)	DOM_DIRECT_VALUE_INJECTION flag fires immediately
TC-09	Candidate opens DevTools to attempt to disable scripts	DevTools detector (3.7-3.8)	Session auto-pauses within 1 second of DevTools open
TC-10	Bluetooth earpiece + accomplice voice relay (simulated)	Hand-to-ear gesture (5.3) + secondary voice detection	CRITICAL fusion event (N=5) fires within 2 seconds of pattern
TC-11	Bezel-mounted phone showing ChatGPT responses	Whitespace fixation (5.2) + shoulder/posture analysis	HIGH fusion event (N=4) fires within 10-second window
TC-12	Honest candidate with ADHD (natural gaze wandering)	KS-test personal baseline (AEGIS ULTRA 5.11) — should NOT flag	Zero flags raised; false positive rate <2%

TC-12 IS MANDATORY: every countermeasure test suite must include honest-candidate edge cases (ADHD, anxiety, medical tremor, cultural eye-contact differences) and verify ZERO false flags. A system that blocks every cheat but also flags honest candidates is not production-ready — false positive testing is not optional.

SECTION 8

DEPLOYMENT CHECKLIST — PRODUCTION LAUNCH GATE

CATEGORY	CHECKLIST ITEM	STATUS GATE
Extension Defense	chrome.management enumeration implemented + consent flow tested	MUST PASS before launch
Extension Defense	Extension blocklist database connected, updates daily	MUST PASS before launch
Extension Defense	Kiosk-mode desktop client available for high-stakes interviews	RECOMMENDED for enterprise tier
DOM/Network	Clipboard, DOM-value, and MutationObserver interceptors all tested (TC-07, TC-08)	MUST PASS before launch
DOM/Network	WebSocket LLM-endpoint blocklist current and pattern-matching enabled	MUST PASS before launch
DOM/Network	DevTools detection auto-pause tested across Chrome, Edge, Firefox, Safari	MUST PASS before launch
Camera/Liveness	PLR+rPPG combined liveness check passes TC-04 (virtual camera test)	MUST PASS before launch
Camera/Liveness	Camera hardware metadata verification against 5+ known virtual cam drivers	MUST PASS before launch
Gesture/Eye/Posture	All 5 fusion channels (gaze, gesture, posture, audio, content) operational	MUST PASS before launch
Gesture/Eye/Posture	KS-test personal baseline calibration completes within 90 seconds reliably	MUST PASS before launch
Gesture/Eye/Posture	Whitespace fixation detector tuned and tested against TC-01, TC-11	MUST PASS before launch
Fusion Engine	N≥3 fusion threshold validated against full TC-01 to TC-11 test matrix	MUST PASS before launch
Fusion Engine	Cross-detection pairs (Section 5.3) all wired and firing correctly	MUST PASS before launch
False Positives	TC-12 honest-candidate edge cases tested: <2% false positive rate achieved	MUST PASS before launch
False Positives	Accommodation Mode tested with documented medical condition scenarios	MUST PASS before launch
Legal/Compliance	EU AI Act Art. 5(1)(f): zero emotion/personality scoring confirmed in codebase	MUST PASS before launch
Legal/Compliance	BIPA: facial embeddings hashed, never stored raw, in production DB schema	MUST PASS before launch
Legal/Compliance	90-day auto-purge Lambda/cron job tested and verified functioning	MUST PASS before launch
Legal/Compliance	Candidate consent forms list every monitored signal explicitly	MUST PASS before launch
Human Review	Ghost View overlay correctly maps every flag type to its visual evidence	MUST PASS before launch
Human Review	Mandatory human review enforced for all N≥4 severity events in workflow	MUST PASS before launch
Performance	CV pipeline sustains 30 FPS on minimum-spec hardware (Intel i5/Ryzen 5, 8GB RAM)	MUST PASS before launch
Performance	Telemetry bandwidth stays under 50 KB/s per session under full load	MUST PASS before launch

AEGIS SHIELD // v1.0 FINAL // June 2026 Blocking Every Bypass Extension + Full Gesture Integration Companion module to AEGIS ULTRA Edition II

Tushar Kapoor B.Tech CSE (AI/ML Specialization)
K.R. Mangalam University, Gurugram GO-BRICS
Business Lab Cohort-I github.com/tusharkapoor052
tusharkapoor052@gmail.com